17

BÖLÜM: ÖNBELLEKLEME MIMARISI VE CDN STRATEJILERI | MODULE: DISTRIBUTED CACHING ARCHITECTURES & CDN STRATEGIES

Önbellek Felaketleri: Cache Avalanche, Stampede (Thundering Herd) ve Penetration

Cache Catastrophes: Avalanche, Stampede (Thundering Herd) & Penetration

📐 Sistem Mimarisi & Veri Akış Şeması

📐 Architecture & Data Flow Diagram

```
[ 1. CACHE AVALANCHE (ÇIĞ ETKİSİ) ]
Milyonlarca anahtarın TTL'i aynı anda (ör. gece 00:00) biter!
Önbellek ⟶ %100 Boşalır ⟶ Milyonlarca İstek Veritabanına Hücum Eder ⟶ DB ÇÖKER!
Çözüm: TTL değerine rastgele Jitter (± 5 dakika) ekleyin!

[ 2. CACHE STAMPEDE / THUNDERING HERD (SÜRÜ İZDIHAMI) ]
Çok popüler tek bir anahtarın (Messi'nin profili) TTL'i biter.
Saniyede 50.000 istek aynı anda DB'ye koşup o tek satırı hesaplamaya çalışır!
Çözüm: Dağıtık Kilit (Mutex / Redlock) veya Arka Planda Proaktif Yenileme!

[ 3. CACHE PENETRATION (DELME SALDIRISI) ]
Saldırgan DB'de OLMAYAN rastgele ID'ler ister (ID: -9999999).
Cache Miss ⟶ DB sorgusu (Bulunamadı) ⟶ Cache'e yazılmaz ⟶ Saldırı DB'yi boğar!
Çözüm: Bloom Filter veya Boş Sonucu (Null Value) kısa TTL ile önbelleğe alma!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bu üç felaketi lokantada anlayalım: 1) Çığ Etkisi (Avalanche): Mutfaktaki tüm hazır yemeklerin aynı anda saat 12:00'de bozulup çöpe atılmasıdır; kapıdaki 500 müşteri doğrudan ocağın başına üşüşür ve mutfak yanar! 2) Sürü İzdihamı (Stampede): Menüdeki en sevilen tatlı biter; 1.000 müşteri aynı anda garsona 'Hemen aşçıya söyle bu tatlıyı yapsın' der, aşçı 1.000 kez aynı tatlıyı pişirmeye çalışırken bayılır! 3) Delme Saldırısı (Penetration): Kötü niyetli biri garsona 'Menüde olmayan Ejderha Çorbası var mı?' diye 100.000 kez sorar. Garson her seferinde depoya gidip bakar, depoyu kilitler!

🇬🇧 ENGLISH (ELI5)

Three restaurant nightmares: 1) Cache Avalanche: Every prepared buffet dish expires at precisely 12:00 PM; a swarm of 500 hungry diners surges straight into the kitchen, suffocating the stoves! 2) Cache Stampede (Thundering Herd): The signature chocolate cake runs out; 10,000 diners concurrently order the chef to bake a new cake, suffocating the pastry station! 3) Cache Penetration: A malicious troll repeatedly asks: 'Do you have Dragon Steak?' (an item that doesn't exist). The waiter marches into the basement warehouse 50,000 times, grinding service to a halt!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

1) Cache Avalanche: Yüzbinlerce önbellek anahtarının aynı TTL ile süresinin dolması sonucu isteklerin doğrudan veritabanını çökertmesidir. Çözüm: TTL süresine rastgele gürültü (Jitter) eklemek (`TTL = 3600 + rand(0, 300)`). 2) Cache Stampede: Yüksek QPS'li tek bir sıcak anahtarın süresi dolduğunda binlerce eşzamanlı thread'in DB'ye hücum etmesidir. Çözüm: Dağıtık Kilit (Mutex): Sadece ilk thread DB'ye gider, diğerleri bekler veya proaktif XFetch algoritması. 3) Cache Penetration: Veritabanında bulunmayan anahtarların sorgulanmasıyla cache'in delinip DB'nin boğulmasıdır. Çözüm: A) Bloom Filter: Olasılıksal veri yapısıyla elemanın DB'de kesinlikle olmadığını $O(1)$ sürede bellekten söyleyerek isteği reddeder. B) Null Caching: Boş sonucu kısa bir TTL (60 sn) ile önbelleğe almak.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

1) Cache Avalanche: Mass simultaneous TTL expirations collapse the database under an instantaneous traffic waterfall. Mitigation: Inject stochastic TTL jitter (`ttl = base_ttl + random_jitter`). 2) Cache Stampede (Thundering Herd): A hyper-popular hot key expires, triggering thousands of concurrent cache misses that hammer the database to compute the identical payload. Mitigation: Mutex locking (only one thread computes, others poll) or probabilistic early expiration (XFetch algorithm). 3) Cache Penetration: Adversaries query non-existent keys (e.g., negative UUIDs), bypassing cache and degrading backend disk IO. Mitigation: Deploy a space-efficient Bloom Filter at the gateway to reject non-existent keys in O(1) time, or cache `NULL` payloads with transient TTLs.

⚡ Modern Java 25 Uygulaması (Loom & Records) ⚡ Java 25 Implementation

```
// Java 25: Cache Stampede (Mutex Kilit) & Avalanche (Jitter) Çözümleri
import java.util.concurrent.*;
import java.util.concurrent.locks.ReentrantLock;

public class CacheDefenses {
    private final ConcurrentMap<String, String> cache = new ConcurrentHashMap<>();
    private final ConcurrentMap<String, ReentrantLock> locks = new ConcurrentHashMap<>();

    // 1. Avalanche Koruması: Rastgele Jitter ile TTL Üretimi
    public int calculateTtlWithJitter(int baseTtlSeconds, int jitterRangeSeconds) {
        int jitter = ThreadLocalRandom.current().nextInt(jitterRangeSeconds);
        return baseTtlSeconds + jitter; // Örn: 3600 + [0-300] sn
    }

    // 2. Stampede Koruması: Mutex Kilit ile Tekil DB Sorgusu
    public String getWithMutex(String key) {
        String val = cache.get(key);
        if (val != null) return val;

        ReentrantLock lock = locks.computeIfAbsent(key, k -> new ReentrantLock());
        lock.lock();
        try {
            // Çift Kontrol (Double-Checked Locking)
            val = cache.get(key);
            if (val != null) return val;

            // Yalnızca TEK bir thread DB'ye gider!
            val = "HEAVY_DB_RESULT_FOR_" + key;
            cache.put(key, val);
            return val;
        } finally {
            lock.unlock();
        }
    }
}

void main() {
    var defense = new CacheDefenses();
    IO.println("Jitter'lı TTL: " + defense.calculateTtlWithJitter(3600, 300) + " saniye");
    IO.println("Stampede Korumalı Sonuç: " + defense.getWithMutex("hot:product:101"));
}
```

⚠️ Mülakat Tuzağı & Darboğaz Uyarısı ⚠️ Interview Trap & Bottleneck Alert

🇹🇷 MÜLAKAT TUZAĞI VE DARBOĞAZ

Mülakatta 'Bloom Filter her şeyi kesin bilir' demeyin! Bloom Filter'ın doğasında 'False Positive' (aslında yokken var diyebilme) payı vardır; ancak 'False Negative' (varken yok deme) riski sıfırdır. Bu ayrımı vurgulamak mülakatçıyı büyüler.

🇬🇧 INTERVIEW TRAP & BOTTLENECK

Never claim Bloom Filters are 100% accurate. Bloom Filters exhibit false positives (may report an item exists when it doesn't), but zero false negatives. Articulating this trade-off marks architectural mastery.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• TTL sürelerine mutlaka rastgele Jitter ekleyerek Çığ Etkisini (Avalanche) önleyin.

• Sıcak anahtarlar için Mutex kilidi kullanarak Sürü İzdihamını (Stampede) engelleyin.

• Bloom Filter ve Null Caching ile veritabanını Delme Saldırılarına (Penetration) karşı koruyun.

🎯 Key Takeaways for the Interview

• Inject stochastic jitter to key TTLs to eliminate concurrent cache avalanche cascades.

• Use distributed mutex locks to ensure only one thread recomputes a hot expired key.

• Shield databases from penetration attacks using in-memory Bloom Filters and Null Caching.